debugging_vit

October 6, 2026

[2]:
```
import json
import sys
from pathlib import Path

# Misma convención que el resto de notebooks/: vive 2 niveles bajo la raíz del proyecto.
project_root = Path.cwd().resolve().parents[1]
print(f"Project root: {project_root}")
sys.path.append(str(project_root))

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn.functional as F
import torchvision

from modules.model import ViTForClassfication
from modules.datasets import CIFAR10Dataset

torch.manual_seed(0)
```

```
Project root: /Users/valefeve/Projects/interpretability of neural networks/interpretable-transformers
```

[2]:
```
<torch._C.Generator at 0x1251364f0>
```

[4]:
```
EXPERIMENT_NAME = "vit-with-15-epochs-CIFAR10"
exp_dir = project_root / "experimentation" / "experiments" / EXPERIMENT_NAME

with open(exp_dir / "config.json") as f:
    config = json.load(f)
with open(exp_dir / "metrics.json") as f:
    metrics = json.load(f)

assert not config["use_faster_attention"], (
    "Este notebook asume block.attention.heads (cabezas separadas); "
    "con use_faster_attention=True habria que adaptar la ablacion por cabeza."
)
```

```
model = ViTForClassfication(config)
state_dict = torch.load(exp_dir / "model_final.pt", map_location="cpu")
model.load_state_dict(state_dict)
model.eval()

n_blocks = len(model.encoder.blocks)
n_heads = config["num_attention_heads"]
print(f"Checkpoint: {EXPERIMENT_NAME}  (accuracy de entrenamiento:␣
 ↪{metrics['accuracy']:.4f})")
print(f"Bloques: {n_blocks}  |  cabezas por bloque: {n_heads}")
```

```
Checkpoint: vit-with-15-epochs-CIFAR10  (accuracy de entrenamiento: 0.6023)
Bloques: 4  |  cabezas por bloque: 4
```

[5]:
```
dataset = CIFAR10Dataset()
dataset.testset = torchvision.datasets.CIFAR10(
    root=str(project_root / "experimentation" / "data"),
    train=False,
    download=True,
    transform=dataset.test_transform,
)
classes = dataset.classes

N_EVAL = 500  # tamano del lote de evaluacion (CPU-friendly)
eval_images, eval_labels = dataset.get_samples_from_indices(range(N_EVAL),␣
 ↪device="cpu", set="test")
print(f"Lote de evaluacion: {N_EVAL} imagenes de test")
```

```
Lote de evaluacion: 500 imagenes de test
```

[8]:
```
with torch.no_grad():
    direct_logits, _ = model(eval_images, output_attentions=False)
```

[ ]:
```
```